Review: ML as a sequence of decisions

Fraida Fund

Making decisions

Building a machine learning system means making a sequence of decisions about the data, the model, and the evaluation. Each decision should be driven by facts: what you know about the data, about the problem, and about how the model will be used. When we make a decision badly, it shows up as a specific type of failure:

- **Failure to fit**: the model has high error even on the *training* data. It cannot represent the relationship between features and target (high bias), or it was not trained well.
- **Failure to generalize**: the model has low error on the training data, but high error on held-out validation/test data. It has learned patterns that are specific to the training sample, including noise (high variance, overfitting).
- **Failure to deploy**: the model does well on the test set, but poorly once it is used for real. The evaluation was overly optimistic, because the test data did not resemble the deployment setting.
- **Failure to generate value**: the model does well on our metric, but does not achieve the goal we actually care about.

and some decisions (e.g. whether we may use some data at all) can cause harm, legal or ethical problems, even if the model "works".

For each decision, the third column lists where it was discussed in the lecture handouts (handout name, section title), the relevant homework problems (HW number, problem number), and the relevant lab problems (Lab number, problem number).

Data

Decision to make	Consequence of a bad decision	Where we saw it
Should we use ML for this problem at all? Or would a rule-based program, or a human expert, be a better fit? ML is a good candidate when there is good data, the thing we want to predict is measurable and observable, and human expertise is insufficient, expensive, or hard to explain.	**Failure to generate value / harm.** If there is an accurate and simple algorithm, an ML model adds cost and error for no benefit. If there is no good data, the model has nothing to learn from. If the problem requires empathy, creativity, insight, or output that we must audit and explain, or if the model may introduce or exacerbate bias, then an ML system may do harm even when its error is low.	**Notes:** *Intro to ML*: "Rule-based vs. data driven problem solving", "Problems that may not be well suited to ML", "Problems that are often good candidates for ML". **HW:** HW1 Q1 (rule-based vs. ML vs. human expert).
Is it legal and ethical to use this data? Consider consent, privacy, copyright, and bias/fairness.	**Harm, legal or ethical failure.** Data collected for one purpose cannot automatically be reused for another; using it anyway can harm the people in the data, lead to lawsuits or regulatory action, and get the system shut down. Data that reflects biased outcomes can produce a model with unequal outcomes, even if it is accurate.	**Notes:** *Working with Data*: "Legal and ethical concerns". **HW:** HW1 Q3 (data use concerns).
Does the data have a signal? Is there a defensible reason the features should predict the target? Could the model use an accidental artifact or proxy instead?	**Failure to fit**, if there is no real signal: no model will beat prediction by the mean. **Failure to generalize**, if a flexible model fits "patterns" in the noise: it can look good on training data, and (if we overfit badly) even on a small validation set, but these patterns do not hold for new data. **Failure to deploy**, if the model learns an artifact of how the data was collected (e.g. candid vs. studio photos) that does not exist in deployment.	**Notes:** *Working with Data*: "Predictive feature concerns", "Example: distinguish real headshots from AI generated headshots"; *Error decomposition, bias-variance tradeoff, model selection*: "Sources of prediction error". **HW:** HW1 Q3 (data use concerns), HW1 Q4 (valid features), HW4 Q13-14 (Parkinson's speech tests). **Lab:** L1 Q1-2 (set expectations, explore relationships), L1 Q5 (baseline).

Decision to make	Consequence of a bad decision	Where we saw it
Is the data representative of the deployment setting? Compare population, context/setting, time period, and target distribution.	**Failure to deploy.** The model learns from, and is evaluated on, a population/setting/time period that differs from where it will be used, so a good test score does not predict deployment performance. Filtering or selecting samples (e.g. only homes with complete records, only offers that sellers accept) also changes what the data represents.	**Notes:** *Working with Data*: "Representativeness concerns", "Example: distinguish real headshots from AI generated headshots". **HW:** HW1 Q3 (data use concerns), HW1 Q8 (train/test/deploy). **Lab:** L3 Q1 (prepare data: filters remove homes with missing measurements), L3 Q3 and Q6 (MSE/pinball reflection: accepted homes differ from all homes), L3 Q8 (Zillow Offers), L4 Q8 (forecast horizon).
Do the data match our expectations, and is the data quality good enough? Before looking at the data, read the data dictionary and write down what you expect to see; then check it with plots, summary statistics, and individual samples. Look for outliers, label errors, data entry errors, duplicate records, and inconsistent units or formats, and decide (for each one) whether it is a true value or an error.	**Failure to fit**, if errors stay in the data: wrong labels and corrupted feature values add noise, and a few extreme values can dominate a squared error loss. **Failure to deploy**, if we "clean" the data without a good reason: removing outliers that are real values (or removing a whole kind of sample, e.g. all low-priced sales) skews the data, so the model and its evaluation no longer reflect the deployment setting.	**Notes:** *Working with Data*: "Make and check assumptions", "Other data quality concerns". **HW:** HW3 Q6-7 (outliers in the data, Huber loss). **Lab:** L1 Q1-2 (set expectations, explore relationships), L1 Q3 (check that data loaded correctly, check for duplicate hours), L2 Q1 (inspect trip-level data), L3 Q1 (convert measurements to numbers, screen out nominal sale prices and inconsistent records).
How should we split the data? Shuffle split, group split, or time-based split, so that the relationship between training and test data matches the relationship between training data and deployment data.	**Failure to deploy.** If the split makes the evaluation task easier than the deployment task (e.g. random split of a time series, or the same person in both training and test sets when the model will be used on new people), the test score is overly optimistic.	**Notes:** *Working with Data*: "Split data"; *Error decomposition, bias-variance tradeoff, model selection*: "K-fold CV - how to split?". **HW:** HW1 Q5 (data split strategy), HW4 Q13-14 (Parkinson's speech tests: `KFold` vs. `GroupKFold`). **Lab:** L1 Q3-4 (load, split, clean, encode; reflection), L2 Q2 (chronological split), L3 Q1 (hold out later sales), L4 Q3 (fixed chronological split), L4 Q5 (time series cross validation).

Decision to make	Consequence of a bad decision	Where we saw it
Which data do we use to compute preprocessing statistics? (e.g. imputation values, encoders, vocabularies, standardization)	**Failure to deploy.** Anything "learned" from the data (a mean, a set of categories, a target encoding, a scaler) must be learned from the training set only, *after* splitting. Otherwise, information from the test set leaks into training, and the evaluation is overly optimistic.	**Notes:** *Working with Data*: "Convert to numeric types" (target encoding, bag of words vocabulary); *One standard error rule, regularization*: "Standardization". **HW:** HW2 Q11 (target encoding), HW4 Q11 (standardize after split). **Lab:** L1 Q3 (process data after splitting), L3 Q1 (fit category encoder on training sales), L4 Q6 (fit `StandardScaler` inside each fold).
How do we handle missing values? First find them (`NaN`, special values like 0 or -1, and rows that are absent entirely), then understand *why* they are missing (MCAR, MAR, MNAR), then leave as missing, drop, or impute, so that the data represents the true (unknown) values as well as possible.	**Failure to deploy / failure to fit.** Dropping rows that are not missing completely at random skews the dataset (e.g. dropping all freezing days because temperature is missing below 32°F), so the model sees too few of some kinds of samples. Imputing a poor value (e.g. the mean, when we know the missing values are extreme) adds error to the feature. A missing value encoded as a "real" value (e.g. 0) silently corrupts the feature.	**Notes:** *Working with Data*: "Handle missing data", "Other data quality concerns". **HW:** HW1 Q6 (missing data). **Lab:** L1 Q3-4 (missing weather values and missing dates; reflection), L3 Q1 (missing measurements), L4 Q2 (missing mobility values).
How do we represent each feature numerically? Ordinal encoding (ordered categories), one-hot encoding (unordered categories), target encoding (categories with many values), bag of words/count vectorizer (text).	**Failure to fit**, if the encoding imposes a structure that is not there (e.g. ordinal encoding of an unordered category forces the model to treat `spicy` as "between" `floral` and `woody`). **Failure to generalize**, if one-hot encoding a feature with many categories creates many sparse features. **Failure to deploy**, if test/deployment data has categories or words not seen in training and we have not handled them, or if target encoding uses test data.	**Notes:** *Working with Data*: "Convert to numeric types". **HW:** HW1 Q7 (one-hot vs. ordinal), HW2 Q11 (target encoding). **Lab:** L1 Q3-4 (one-hot encode weather summary, align training and test columns; reflection), L3 Q1 (one-hot encode borough, property type, neighborhood, building class).

Model

Decision to make	Consequence of a bad decision	Where we saw it
Define what one sample is (the unit of prediction): one person, one trip, one hour, one sale, one borough on one day... It should match what the model will be asked to predict in deployment.	**Failure to generate value**, if the sample doesn't match the prediction we need (e.g. the data has one row per bike trip, but we need to predict the number of trips per hour, so we must aggregate first). **Failure to deploy**, if the same entity appears in many samples and we don't account for it (e.g. many recordings from the same patient) when splitting the data.	**Notes:** *Working with Data*: "Construct samples"; *Intro to ML*: "The basic supervised learning problem". **HW:** HW1 Q9 (feature matrix and target vector), HW4 Q13-14 (Parkinson's speech tests: many recordings per patient). **Lab:** L2 Q1 (aggregate trips per hour), L3 Q1 (one sample is a sale, but an assessment describes a parcel), L4 Q2 (one sample per borough per day).
Match features and target for each sample. When combining data sources, join them on a key that gives a correct one-to-one (or many-to-one) correspondence, and check that the join did what you expected.	**No supervised learning possible**, if the features and the target are never observed together for the same samples. **Failure to fit**, if the join is wrong (e.g. duplicate keys, mismatched dates or locations): the features no longer describe the sample they are attached to, so they become noise. **Failure to deploy**, if a joined feature describes a time *after* the prediction would be made.	**Notes:** *Working with Data*: "Construct samples". **HW:** HW1 Q3 (data use concerns: features and target not observed together). **Lab:** L3 Q1 (join assessments to sales; duplicate parcel IDs), L4 Q2 (join case rates and weather to mobility data; validate the keys).
Choose the target variable. It should be measurable, available, and correct, and it should match the goal of the system.	**Failure to generate value**, if the target is a poor proxy for what we actually care about (e.g. predicting healthcare *cost* when we care about healthcare *need*): the model can be accurate and still not achieve the goal. **Failure to fit**, if the target is noisy or mislabeled.	**Notes:** *Working with Data*: "Select target", "Other data quality concerns"; *Intro to ML*: "Goal of a supervised learning system". **HW:** HW1 Q2 (target variable for mental health system), HW1 Q9 (feature matrix and target vector). **Lab:** L2 Q1 (aggregate trips per hour), L4 Q2 (transit target).

Decision to make	Consequence of a bad decision	Where we saw it
Choose the features. Include features that are plausibly predictive and available at inference time; exclude identifiers, irrelevant fields, and fields that won't be available when the model is used.	**Failure to fit**, if there are no good features (the model is missing information it needs, so it is biased). **Failure to generalize**, if there are too many features for the amount of data, so the model fits noise (high variance). **Failure to deploy**, if we use a *leaky* feature that is not available at inference time, or is a proxy for the target in the data but not in deployment.	**Notes:** *Working with Data*: "Select features"; *Error decomposition, bias-variance tradeoff, model selection*: "Bias of linear model", "Variance of linear model". **HW:** HW1 Q3 (data use concerns), HW1 Q4 (valid features), HW4 Q1 (what to try next), HW4 Q6-7 (one-SE rule for number of features). **Lab:** L1 Q1 (variables that should not be used), L1 Q4 (feature selection reflection), L2 Q4-5 (weekend and rush hour features), L4 Q3-4 (same-day features, lag features), L4 Q8 (forecast horizon: same-day information is not known at forecast time).
Choose the model class. So far: a linear model $\widehat{y} = \langle w, \phi(x) \rangle$, compared against prediction by the mean.	**Failure to fit**, if the true relationship is not in the model class (e.g. a linear model in x for a non-linear relationship): the model is biased no matter how much data we have.	**Notes:** *Linear Regression*: "Simple linear regression model", "Multiple linear regression", "Understanding the coefficients - multiple regression"; *Error decomposition, bias-variance tradeoff, model selection*: "Bias of linear model". **HW:** HW2 Q1 (graphical intuition), HW2 Q6 (compute fitted value), HW2 Q10 (walkability), HW4 Q2 (model class), HW4 Q3 (bias and variance of two models). **Lab:** L1 Q5 (baseline), L2 Q2 (linear regression on hour).
Choose the loss function to match the problem. Mean squared error (fits the conditional mean), Huber loss (less sensitive to outliers), pinball loss (fits a quantile, for asymmetric costs of over- vs. under-predicting).	**Failure to generate value**, if the loss doesn't reflect what errors actually cost (e.g. an MSE house price model has a good R^2 but loses money, because over-estimating a price is much more costly than under-estimating it). **Failure to fit** the bulk of the data, if a few outliers dominate a squared error loss.	**Notes:** *Linear Regression*: "Mean squared error loss function", "Least squares loss for multiple/LBF regression"; *Gradient descent*: "Solution using gradient descent" (Huber and pinball losses are covered in the HW and lab, not the handouts). **HW:** HW3 Q5 (Huber update rule), HW3 Q6 (gradient descent step with Huber loss), HW3 Q7 (Huber loss in Python). **Lab:** L3 Q2-3 (MSE model; reflection), L3 Q4 (pinball update rule), L3 Q5-6 (pinball model; reflection), L3 Q7 (market simulation), L3 Q8 (Zillow Offers).

Decision to make	Consequence of a bad decision	Where we saw it
Regularize? Add a penalty on the size of the weights (Ridge/L2, LASSO/L1) and choose its strength α.	**Failure to generalize**, with too little regularization when there are many (or correlated) features: large, unstable weights. **Failure to fit**, with too much regularization: weights are shrunk too far toward zero, and training error increases.	**Notes:** *One standard error rule, regularization*: "Penalty for model complexity", "Regularization vs. standard LS", "Common regularizers", "Effect of regularization level", "Selecting regularization level". **HW:** HW4 Q4 (model comparison), HW4 Q5 (training and test MSE vs. α), HW4 Q9 (one-SE rule for α), HW4 Q12 (Ridge for diamond prices). **Lab:** L4 Q6 (choose regularization strength with time series CV), L4 Q7 (reflection: Ridge vs. least squares with 180 lags).
Fit the model. Choose a solver (closed-form OLS or gradient descent), an optimizer (batch, mini-batch, stochastic; momentum, Adam, etc.), and training hyperparameters (learning rate, batch size, number of iterations); standardize features if needed.	**Failure to fit.** If the learning rate is too small or we stop too early, the weights have not converged; if it is too large, the loss oscillates or diverges. Features on very different scales make the loss surface a "ravine", so gradient descent converges slowly (and penalties treat the features unequally). Check the loss curve!	**Notes:** *Linear Regression*: "Ordinary least squares solution for multiple/basis function regression"; *Gradient descent*: "Standard ("batch") gradient descent", "Stochastic gradient descent", "Mini-batch (also"stochastic") gradient descent", "Selecting the learning rate", "Annealing the learning rate", "Gradient descent in a ravine", "Momentum", "Adam"; *One standard error rule, regularization*: "Standardization". **HW:** HW2 Q9 (design matrix), HW3 Q1 (SGD, graphically), HW3 Q2 (visualize learning rate), HW3 Q3 (batch size), HW3 Q4 (gradient descent step). **Lab:** L3 Q2 (implement gradient descent, inspect loss curve), L3 Q5 (scale features and prices before fitting).
Use basis function transformations. Inspect the residuals (vs. each feature, and over time) to see what the model is missing, then transform features to match.	**Failure to fit**, if a systematic pattern remains in the residuals: the model cannot represent the relationship using the features as given. (But adding many basis functions makes the model more flexible, and can cause **failure to generalize**, especially when extrapolating.)	**Notes:** *Linear Regression*: "Linear basis function regression", "Residual term"; *Working with Data*: "Create"transformed" features". **HW:** HW2 Q4 (feature transformations), HW2 Q5 (residual plots → feature engineering). **Lab:** L2 Q2 (residuals of raw hour model), L4 Q1 (the "cubic model").

Decision to make	Consequence of a bad decision	Where we saw it
– *Shapes* like $\log x$, e^x, $\sqrt{x}$, polynomials	Use when the residuals show a curved trend in one feature.	**HW:** HW2 Q4 (feature transformations), HW2 Q5 (residual plots), HW2 Q7 (log-transformed prediction), HW2 Q12 (baby size), HW2 Q13-14 (basis functions). **Lab:** L3 Q1 (log of area), L4 Q1 (polynomial).
– *Splines* (piecewise polynomials joined at knots)	Use for a non-linear shape that changes across the range of x; number of knots and degree is a model selection problem.	**Notes:** *Error decomposition, bias-variance tradeoff, model selection*: "Model selection problems".
– *Fourier* features (sin, cos of a cycle)	Use for cyclic features (hour of day, month of year), where the "end" of the cycle is next to the "start".	**HW:** HW2 Q5 (residual plots). **Lab:** L2 Q3 (24-hour and 12-hour cycles), L3 Q1 (sale month).
– *Indicators and interactions*	Use when the effect of a feature is different for different groups (e.g. weekday vs. weekend), or concentrated in a range (e.g. rush hour).	**HW:** HW2 Q5 (residual plots), HW2 Q8 (interaction terms), HW2 Q9 (design matrix with interaction and indicator). **Lab:** L2 Q4 (weekend interaction), L2 Q5 (rush hour indicator).
– *Autoregressive lag* features	Use for time series where recent values of the target predict the next value (then, residuals over time show what is still missing).	**HW:** HW4 Q8 (one-SE rule for number of lags). **Lab:** L4 Q4 (lags 1-7), L4 Q5 (choose lag order), L4 Q7 (reflection).
Use cross validation to compare models and choose one (e.g. number of features, polynomial order, number of lags, α), with the one-SE rule to prefer a simpler model when the difference is within the noise.	**Failure to fit**, if we choose a model that is too simple. **Failure to generalize**, if we choose by training error, or choose the model with the very best validation score when a simpler model is just as good. **Failure to deploy**, if we choose using the test set (it is no longer an independent estimate), or use CV folds that don't match deployment (e.g. shuffled folds for a time series or grouped data).	**Notes:** *Error decomposition, bias-variance tradeoff, model selection*: "Bias-variance tradeoff", "Model selection problems", "Hold-out validation", "Problems with hold-out validation", "K-fold cross validation", "K-fold CV - how to split?"; *One standard error rule, regularization*: "One-SE rule", "Selecting regularization level". **HW:** HW4 Q1 (what to try next), HW4 Q6-9 (one-SE rule), HW4 Q10 (one-SE rule in code), HW4 Q13-14 (Parkinson's speech tests). **Lab:** L4 Q5 (choose lag order with time series CV), L4 Q6 (choose α with time series CV), L4 Q7 (reflection).

Evaluation

Decision to make	Consequence of a bad decision	Where we saw it
What do we compare against? A simple baseline, e.g. prediction by the mean ($R^2 = 0$), or "same value as last week".	**Undetected failure to fit.** Without a baseline, we can't tell whether the model learned anything from the features. A negative R^2 means the model is worse than predicting the mean.	**Notes:** *Linear Regression*: "Prediction by mean", "Interpreting R2 as error relative to"mean model"". **HW:** HW1 Q10 (prediction by mean), HW2 Q1 (graphical intuition, positive and negative R^2), HW4 Q13-14 (Parkinson's speech tests). **Lab:** L1 Q5 (`DummyRegressor` baseline), L4 Q3 (same-day model with negative test R^2), L4 Q8 (same-weekday baseline).
Which metric? MSE/RMSE, MAE, MAPE, R^2; choose the one that reflects what errors actually cost in the application.	**Failure to generate value**, if we optimize and report a metric that doesn't reflect the goal: e.g. a model with better R^2 can lose more money. Different metrics weigh errors differently (squared error emphasizes large errors; MAPE emphasizes errors on small values; a few very expensive homes affect MAE and MAPE very differently).	**Notes:** *Linear Regression*: "Interpreting MSE, MAE", "Interpreting R2 as explained variance", "Interpreting R2 as error relative to"mean model"". **HW:** HW2 Q2 (residuals, variance, and MSE graphically), HW2 Q3 (R^2 vs. coefficient), HW2 Q10 (walkability), HW2 Q11 (MAE vs. MAPE). **Lab:** L2 Q2 (MSE, RMSE, R^2), L3 Q3 and Q6 (valuation metrics vs. P&L), L3 Q8 (Zillow Offers).
Compare training and validation error to diagnose the problem before deciding what to change.	Choosing the wrong fix. Both errors high: **failure to fit** (high bias), so add features, a more flexible model, or less regularization. Training error low but validation error high: **failure to generalize** (high variance), so get more data, use fewer features, a less flexible model, or more regularization. Note that getting more data reduces variance, but does not reduce bias: if the training error is already high, more of the same data won't help.	**Notes:** *Error decomposition, bias-variance tradeoff, model selection*: "When learning fails", "Error decomposition", "Bias-variance tradeoff". **HW:** HW4 Q1 (what to try next), HW4 Q3 (bias and variance of two models), HW4 Q5 (training and test MSE vs. α). **Lab:** L4 Q3-5 (training and test RMSE; validation curve), L4 Q7 (reflection).
Inspect the residuals, not only a summary metric: residuals vs. each feature, vs. prediction, and over time.	**Undetected failure to fit.** A single number can hide a systematic pattern (a missing cycle, a missing group effect, a trend over time) that tells us how to improve the model.	**Notes:** *Linear Regression*: "Residual term". **HW:** HW2 Q2 (residuals graphically), HW2 Q5 (residual plots → feature engineering). **Lab:** L2 Q2-5 (residual plots for each Citi Bike model), L4 Q3-4 (residuals over time), L4 Q7 (reflection).

Decision to make	Consequence of a bad decision	Where we saw it
Keep the test set independent: use it once, for the final evaluation, after model selection and preprocessing were done with training data only.	**Failure to deploy.** If the test set influences preprocessing, training, or model selection, the test score is overly optimistic.	**Notes:** *Working with Data*: "Model training vs evaluation vs deployment", "Split data"; *Error decomposition, bias-variance tradeoff, model selection*: "Model selection problems", "Hold-out validation". **HW:** HW1 Q8 (train/test/deploy), HW4 Q11 (standardize after split). **Lab:** L4 Q5-6 (CV on training data, then evaluate the selected model on the test set).
Does the evaluation reflect deployment? Same kind of samples, same information available at prediction time, same prediction horizon, and the outcome we actually care about.	**Failure to deploy / failure to generate value.** A model that is "best" on training data may not be best on test data, and a model that is best on test data may not be best in deployment. A forecast evaluated using information that won't be known yet (e.g. tomorrow's weather) looks better than it will be.	**Notes:** *Working with Data*: "Model training vs evaluation vs deployment", "Representativeness concerns"; *Intro to ML*: "ML training vs reality". **HW:** HW1 Q8 (train/test/deploy), HW4 Q13-14 (Parkinson's speech tests). **Lab:** L3 Q7 (market simulation), L3 Q8 (Zillow Offers), L4 Q8 (forecast horizon).
How do we turn predictions into actions? A prediction is usually an input to a decision (e.g. offer price = estimated price minus an offset), and the outcome depends on that decision and on how other people respond to it. Evaluate the decision by the outcome we care about (e.g. profit), not only by the prediction error.	**Failure to generate value.** The same model can make or lose money depending on how its predictions are used. When others choose whether to accept our decision, they accept it most often when it is in their favor (e.g. sellers accept offers on homes that the model *over*-estimated), so the outcomes we actually experience can be much worse than the average prediction error suggests.	(Covered in the lab, not the handouts.) **Lab:** L3 Q3 (MSE model: offer offset vs. P&L), L3 Q6 (pinball model: offer offset vs. P&L), L3 Q7 (market simulation), L3 Q8 (Zillow Offers).

Computations

You should be able to do each of these computations by hand (with a calculator), for small examples.

The problems we practiced use specific examples, such as a log or square root transformation, interaction terms, lagged values, MSE, MAE, Huber, or pinball loss. But you should be able to apply the same procedure to *any* basis function transformation, loss function, regularization penalty, or update rule that you are given, including ones we have not seen before.

In the third column, "Practice" refers to the "Extra pre-midterm practice" problem set.

Computation	How to do it	Where we practiced it
Write the feature matrix X and target vector y from a table of data.	One row per sample, one column per feature. Leave out sample identifiers; the target goes in y, not X.	**Notes:** *Intro to ML*: "The basic supervised machine learning problem - matrix form". **HW:** HW1 Q9. **Practice:** Q47.
Compute the output of prediction by mean.	$\hat{y}_i = \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ for every sample, using the target values of the *training* data.	**Notes:** *Linear Regression*: "Prediction by mean". **HW:** HW1 Q10. **Practice:** Q48.
Compute the prediction of a fitted linear model, with any basis function transformation of the input.	First apply each basis function to the input to get $\phi_0(\mathbf{x}), \ldots, \phi_p(\mathbf{x})$ (with $\phi_0(\mathbf{x}) = 1$ for the intercept), then compute the weighted sum $\hat{y} = \sum_j w_j \phi_j(\mathbf{x})$. The transformation can be anything: e.g. $\ln x_j$, $\sqrt{x_j}$, x_j^2, $x_1 x_2$, an indicator such as $\mathbb{1}(x_1 > c)$, sin/cos of a cycle, or lagged values $y_{k-1}, \ldots, y_{k-D}$ of a time series. Be careful to match each weight to the right feature.	**Notes:** *Linear Regression*: "Linear model", "Basis functions", "Linear basis function model for regression", "Vector form of linear basis function model". **HW:** HW2 Q6 (linear), HW2 Q7 (log), HW2 Q8 (interactions). **Practice:** Q49-50 (linear), Q51 (log), Q52 (interactions), Q53 (lagged values), Q54 (polynomial), Q55 (square root).
Write the design matrix Φ (or A) for a linear model with any basis functions, and set up the least squares problem.	One row per sample, one column per basis function: entry (i, j) is $\phi_j(\mathbf{x}_i)$. The first column is all ones if the model has an intercept. Then $\hat{\mathbf{y}} = \Phi\mathbf{w}$, and the least squares weights solve the normal equations $\Phi^T\Phi\mathbf{w} = \Phi^T\mathbf{y}$.	**Notes:** *Linear Regression*: "Matrix form of linear basis function model", "Setup", "Solving multiple/LBF regression". **HW:** HW2 Q9 (interaction and indicator), HW2 Q13 (e^{-jx}), HW2 Q14 (same, in Python). **Practice:** Q56 (with A^T and $\mathbf{y}$), Q57 (interaction and indicator), Q58 (x^{2j}), Q97-98 (in Python). **Lab:** L2 Q3 (Fourier features), L4 Q4 (lag features).
Count gradient computations and weight updates per epoch for full batch, mini-batch, and stochastic gradient descent.	In one epoch, we always compute the gradient of the loss for all N samples. The number of weight updates is the number of batches: 1 for full batch, $\lceil N/B \rceil$ for mini-batch size B, N for stochastic gradient descent.	**Notes:** *Gradient descent*: "Stochastic gradient descent", "Mini-batch (also"stochastic") gradient descent". **HW:** HW3 Q3. **Practice:** Q59.

Computation	How to do it	Where we practiced it
Compute one step of gradient descent by hand, given data, initial weights, a learning rate, and any update rule.	For each sample: compute the prediction $\hat{y}_i$, the residual $y_i - \hat{y}_i$, and the per-sample gradient. Add them up and divide by n (for the samples in this batch), add any penalty gradient, multiply by α, and subtract from $\mathbf{w}^t$. Keep extra decimal places until the end.	**Notes:** *Gradient descent*: "Example: gradient descent for linear regression". **HW:** HW3 Q4 (MSE), HW3 Q6 (Huber). **Practice:** Q60 (MSE), Q61 (MAE), Q62 (MSE with L1 penalty), Q63 (Huber), Q64 (pinball).
Apply the one-SE rule to choose a model.	For each model p, compute the mean score across the K folds $\bar{S}_p$, the standard deviation σ_p, and $SE_p = \frac{\sigma_p}{\sqrt{K-1}}$. Find the best model p^*. For a lower-is-better metric (e.g. MSE), the target is $\bar{S}_{p^*} + SE_{p^*}$ and candidates have $\bar{S}_p \leq$ target; for a higher-is-better metric (e.g. R^2), the target is $\bar{S}_{p^*} - SE_{p^*}$ and candidates have $\bar{S}_p \geq$ target. Then choose the *simplest* candidate: fewest features, smallest lag order, or *largest* regularization strength α.	**Notes:** *One standard error rule, regularization*: "One-SE rule", "One-SE rule: compute standard error", "One-SE rule: Step 1 with lower-is-better metric", "One-SE rule: Step 1 with higher-is-better metric". **HW:** HW4 Q6-10. **Practice:** Q67-71, Q100 (in Python). **Lab:** L4 Q5.

General facts